CSS Counter

Increment & Reset Guide

Create automatic numbering with CSS counters

Easy English • 45+ Examples • Ready to Use

Contents

Chapter 1

Introduction to CSS Counters

1.1 What are CSS Counters?

CSS counters are variables that increment or decrement as you move through your document. They allow you to automatically number elements using pure CSS without adding numbers to your HTML. Counters work in conjunction with `counter-reset`, `counter-increment`, and the `counter()` function.

Think of it as: "Create auto-incrementing numbers using only CSS."

Key Concept

CSS counters are powerful for creating numbered lists, chapter numbers, page numbers, footnote references, and any sequential numbering system without modifying HTML or using JavaScript.

1.2 Three Key Properties

- **counter-reset** - Initialize a counter to a starting value
- **counter-increment** - Increase the counter value
- **counter()** - Display the current counter value

1.3 Why Use CSS Counters?

- Keep HTML clean without hard-coded numbers
- Create dynamic numbering systems
- Number headings, sections, and chapters automatically
- Create sequential references and citations
- Build complex nested numbering (1, 1.1, 1.1.1, etc.)
- No JavaScript needed

- Easy to maintain and modify

1.4 Browser Support

CSS counters have excellent browser support in all modern browsers (Chrome, Firefox, Safari, Edge). They work reliably across all platforms.

Chapter 2

Basic Concepts and Syntax

2.1 1. counter-reset Property

The `counter-reset` property initializes a counter to a specified value (default is 0).

2.1.1 Syntax

```
element {
    counter-reset: counter-name;
    counter-reset: counter-name initial-value;
}
```

2.1.2 Example

CSS Code

```
body {
    counter-reset: chapter;
}

article {
    counter-reset: section;
}
```

2.2 2. counter-increment Property

The `counter-increment` property increases a counter by a specified amount (default is 1).

2.2.1 Syntax

```
element {
    counter-increment: counter-name;
    counter-increment: counter-name increment-value;
}
```

2.2.2 Example

CSS Code

```
h1 {
    counter-increment: chapter;
}

h2 {
    counter-increment: section;
}
```

2.3 3. counter() Function

The `counter()` function displays the current counter value using the `content` property.

2.3.1 Syntax

```
element::before {
    content: counter(counter-name);
    content: counter(counter-name, style);
}
```

2.3.2 Example

CSS Code

```
h1::before {
    content: "Chapter " counter(chapter) ": ";
}

li::before {
    content: counter(item) ". ";
}
```

Chapter 3

Practical Examples

3.1 Example 1: Simple Numbered List

HTML

```
<ol>
    <li>First item</li>
    <li>Second item</li>
    <li>Third item</li>
</ol>
```

]

```
ol {
    list-style: none;
    counter-reset: item;
    padding-left: 0;
}

ol li {
    counter-increment: item;
    margin-bottom: 8px;
    padding-left: 30px;
    position: relative;
}

ol li::before {
    content: counter(item) ".";
    position: absolute;
    left: 0;
    font-weight: bold;
    color: blue;
}
```

]

```
<h1>Chapter Title</h1>
<h2>Section 1</h2>
<h2>Section 2</h2>
```

```
<h1>Another Chapter</h1>
<h2>Section 1</h2>
```

]

```
body {
    counter-reset: chapter;
}

h1 {
    counter-reset: section;
    counter-increment: chapter;
}

h1::before {
    content: counter(chapter) ". ";
}

h2 {
    counter-increment: section;
}

h2::before {
    content: counter(chapter) "." counter(section) " ";
}
```

Result: - ”1. Chapter Title” - ”1.1 Section 1” - ”1.2 Section 2” - ”2. Another Chapter” - ”2.1 Section 1”

3.2 Example 3: Circular Badge Numbers

]

```
<div class="steps">
    <div class="step">Install</div>
    <div class="step">Configure</div>
    <div class="step">Deploy</div>
</div>
```

]

```
.steps {
    counter-reset: step-counter;
}

.step {
    counter-increment: step-counter;
    padding: 15px;
    margin: 10px;
    border-left: 4px solid blue;
}

.step::before {
    content: counter(step-counter);
    display: inline-block;
    width: 30px;
    height: 30px;
    background-color: blue;
    color: white;
    border-radius: 50%;
    text-align: center;
    line-height: 30px;
    margin-right: 10px;
    font-weight: bold;
}
```

3.3 Example 4: Table Row Numbers

]

```
<table>
    <tr>
        <td>Product A</td>
        <td>$99</td>
    </tr>
    <tr>
        <td>Product B</td>
        <td>$149</td>
    </tr>
    <tr>
        <td>Product C</td>
        <td>$199</td>
    </tr>
</table>
```

[colback=codegray, colframe=darkblue, title=CSS]

```
table {
    counter-reset: row-number;
}

table tr {
    counter-increment: row-number;
}

table tr::before {
    content: counter(row-number);
    display: table-cell;
    font-weight: bold;
    width: 30px;
    padding: 8px;
    background-color: #f0f0f0;
}
```

3.4 Example 5: Custom Starting Number

Start a counter at a custom value:

[colback=codegray, colframe=darkblue, title=CSS]

```
/* Start numbering from 10 */
ol {
    counter-reset: item 9;
}

ol li {
    counter-increment: item;
}

ol li::before {
    content: counter(item) ". ";
}

/* Start from 100 */
.special-list {
    counter-reset: special 99;
}

.special-list .item {
    counter-increment: special;
}

.special-list .item::before {
    content: counter(special) " - ";
}
```

Result: - First list item displays as: "10. ..." - Special list items start from: "100 - ..."

Chapter 4

Counter Styles and Formatting

4.1 1. Different Number Styles

CSS counters support various numbering styles:
[colback=codegray, colframe=darkblue, title=CSS]

```
/* Decimal (default) */
li::before { content: counter(list, decimal); }

/* Leading zeros */
li::before { content: counter(list, decimal-leading-zero); }

/* Lowercase letters */
li::before { content: counter(list, lower-alpha); }

/* Uppercase letters */
li::before { content: counter(list, upper-alpha); }

/* Lowercase Roman numerals */
li::before { content: counter(list, lower-roman); }

/* Uppercase Roman numerals */
li::before { content: counter(list, upper-roman); }

/* CJK symbols */
li::before { content: counter(list, cjk-ideographic); }

/* Armenian */
li::before { content: counter(list, armenian); }

/* Georgian */
li::before { content: counter(list, georgian); }
```

4.2 2. Visual Examples

Style	Output for 1,2,3
decimal	1, 2, 3
decimal-leading-zero	01, 02, 03
lower-alpha	a, b, c
upper-alpha	A, B, C
lower-roman	i, ii, iii
upper-roman	I, II, III

4.3 3. Example: Mixed Numbering Styles

[colback=codegray, colframe=darkblue, title=HTML]

```
<h2>Chapter 1</h2>
<h3>Section A</h3>
<h3>Section B</h3>
<h2>Chapter 2</h2>
<h3>Section A</h3>
```

[colback=codegray, colframe=darkblue, title=CSS]

```
body {
    counter-reset: chapter;
}

h2 {
    counter-reset: section;
    counter-increment: chapter;
}

h2::before {
    content: "Chapter " counter(chapter, upper-roman) ": ";
}

h3 {
    counter-increment: section;
}

h3::before {
    content: counter(chapter, upper-roman) "."
             counter(section, lower-alpha) " ";
}
```

Result: - ”Chapter I: Chapter 1” - ”I.a Section A” - ”I.b Section B” - ”Chapter II: Chapter 2” - ”II.a Section A”

Chapter 5

Advanced Patterns

5.1 1. Nested Counters

Create multi-level numbering systems:

[colback=codegray, colframe=darkblue, title=HTML]

```
<ol class="outline">
    <li>Item 1
        <ol>
            <li>Sub-item 1.1</li>
            <li>Sub-item 1.2</li>
        </ol>
    </li>
    <li>Item 2
        <ol>
            <li>Sub-item 2.1</li>
        </ol>
    </li>
</ol>
```

[colback=codegray, colframe=darkblue, title=CSS]

```
.outline {
    list-style: none;
    counter-reset: item;
}

.outline li {
    counter-increment: item;
}

.outline li::before {
    content: counter(item) ". ";
}

.outline ol {
    counter-reset: subitem;
    margin-left: 40px;
}

.outline ol li {
```

```
    counter-increment: subitem;
}

.outline ol li::before {
    content: counter(item) "." counter(subitem) " ";
}
```

5.2 2. Multi-Level Counters with counters()

The `counters()` function displays all parent counters:
[colback=codegray, colframe=darkblue, title=CSS]

```
body {
    counter-reset: h1-counter;
}

h1 {
    counter-increment: h1-counter;
    counter-reset: h2-counter;
}

h1::before {
    content: counter(h1-counter) ". ";
}

h2 {
    counter-increment: h2-counter;
    counter-reset: h3-counter;
}

h2::before {
    content: counters(h2-counter, ".") ". ";
}

h3 {
    counter-increment: h3-counter;
}

h3::before {
    content: counters(h3-counter, ".") ". ";
}
```

5.3 3. Incrementing by Custom Values

[colback=codegray, colframe=darkblue, title=CSS]

```
/* Increment by 1 (default) */
li {
    counter-increment: item;
}

/* Increment by 2 */
li {
    counter-increment: item 2;
}

/* Increment by 0.5 */
li {
    counter-increment: score 0.5;
}
```

```
/* Decrement */
li {
    counter-increment: item -1;
}
```

Chapter 6

Real-World Projects

6.1 Project 1: Book with Chapters and Sections

[colback=codegray, colframe=darkblue, title=HTML]

```
<div class="book">
    <h1>Chapter 1: Introduction</h1>
    <h2>Background</h2>
    <p>Content...</p>
    <h2>Overview</h2>
    <p>Content...</p>

    <h1>Chapter 2: Methods</h1>
    <h2>Approach</h2>
    <p>Content...</p>
</div>
```

[colback=codegray, colframe=darkblue, title=CSS]

```
.book {
    counter-reset: chapter;
}

.book h1 {
    counter-increment: chapter;
    counter-reset: section;
    font-size: 2em;
}

.book h1::before {
    content: "Chapter " counter(chapter) ": ";
    color: blue;
}

.book h2 {
    counter-increment: section;
    font-size: 1.3em;
    margin-left: 20px;
}

.book h2::before {
```

```
    content: counter(chapter) "." counter(section) " ";
    color: gray;
}
```

6.2 Project 2: Figure and Table Numbering

[colback=codegray, colframe=darkblue, title=HTML]

```
<article>
    <h1>Report</h1>

    <figure>
        <img src="graph.jpg" alt="Sales">
    </figure>

    <table>
        <tr><td>Data</td></tr>
    </table>

    <figure>
        <img src="chart.jpg" alt="Growth">
    </figure>
</article>
```

[colback=codegray, colframe=darkblue, title=CSS]

```
article {
    counter-reset: figure table;
}

figure {
    counter-increment: figure;
    border: 1px solid #ddd;
    padding: 10px;
}

figure::after {
    content: "Figure " counter(figure);
    display: block;
    text-align: center;
    font-style: italic;
    margin-top: 10px;
}

table {
    counter-increment: table;
    width: 100%;
    border-collapse: collapse;
}

table::before {
    content: "Table " counter(table);
    display: block;
    font-weight: bold;
    margin-bottom: 10px;
}
```

6.3 Project 3: FAQ with Automatic Numbering

[colback=codegray, colframe=darkblue, title=HTML]

```
<div class="faq">
    <div class="faq-item">
        <h3 class="question">How do I get started?</h3>
        <p class="answer">Answer here...</p>
    </div>

    <div class="faq-item">
        <h3 class="question">What's included?</h3>
        <p class="answer">Answer here...</p>
    </div>

    <div class="faq-item">
        <h3 class="question">Need support?</h3>
        <p class="answer">Answer here...</p>
    </div>
</div>
```

[colback=codegray, colframe=darkblue, title=CSS]

```
.faq {
    counter-reset: faq-counter;
}

.faq-item {
    margin: 20px 0;
    border-left: 4px solid blue;
    padding: 15px;
}

.question {
    counter-increment: faq-counter;
}

.question::before {
    content: "Q" counter(faq-counter) ": ";
    font-weight: bold;
    color: blue;
}

.answer::before {
    content: "A" counter(faq-counter) ": ";
    font-weight: bold;
    color: green;
}
```

6.4 Project 4: Tutorial Steps

[colback=codegray, colframe=darkblue, title=HTML]

```
<div class="tutorial">
    <div class="step">
        <h3>Installation</h3>
        <p>Download and install...</p>
    </div>

    <div class="step">
        <h3>Setup</h3>
        <p>Configure your settings...</p>
    </div>

    <div class="step">
        <h3>First Run</h3>
        <p>Launch the application...</p>
    </div>
</div>
```

[colback=codegray, colframe=darkblue, title=CSS]

```
.tutorial {
    counter-reset: step;
}

.step {
    counter-increment: step;
    margin: 20px 0;
    padding: 20px;
    position: relative;
    border-left: 4px solid blue;
}

.step::before {
    content: counter(step);
    position: absolute;
    top: 20px;
    left: -22px;
    width: 40px;
    height: 40px;
    background: blue;
    color: white;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: bold;
}

.step h3 {
    margin-top: 0;
}
```

Chapter 7

Tips and Best Practices

7.1 Tip 1: Always Reset at Parent Level

Reset counters at the appropriate level to avoid conflicts:

```
/* Good - Reset at parent */
ol {
    counter-reset: item;
}

ol li {
    counter-increment: item;
}

/* Avoid - Resetting at every element */
ol li {
    counter-reset: item;
    counter-increment: item;
}
```

7.2 Tip 2: Use Descriptive Counter Names

Choose clear names for counters:

```
/* Good - Clear intent */
counter-reset: chapter section;
counter-reset: faq-question faq-answer;

/* Avoid - Vague names */
counter-reset: c s;
counter-reset: x y;
```

7.3 Tip 3: Reset Nested Counters

When incrementing a parent counter, reset child counters:

```
h1 {
    counter-increment: chapter;
```

```
    counter-reset: section;
}

h2 {
    counter-increment: section;
    counter-reset: subsection;
}

h3 {
    counter-increment: subsection;
}
```

7.4 Tip 4: Use counters() for Hierarchy

Use `counters()` to show full numbering path:

```
h3::before {
    content: counters(heading, ".") " ";
}

/* Displays: 1.2.3 instead of just 3 */
```

7.5 Tip 5: Combine with Content Property

Always use counters with the `content` property:

```
/* Right - Use in content */
li::before {
    content: counter(item) ". ";
}

/* Wrong - Can't use directly */
/* li { counter: item; } */
```

Chapter 8

Common Mistakes to Avoid

8.1 Mistake 1: Forgetting counter-reset

```
/* WRONG - No reset, counter keeps increasing */
li {
    counter-increment: item;
}

li::before {
    content: counter(item);
}

/* RIGHT - Reset at parent */
ol {
    counter-reset: item;
}

ol li {
    counter-increment: item;
}

ol li::before {
    content: counter(item);
}
```

8.2 Mistake 2: Resetting at Wrong Level

```
/* WRONG - Resets at every item */
li {
    counter-reset: item;
    counter-increment: item;
}

/* RIGHT - Reset at parent, increment at child */
ol {
    counter-reset: item;
}
```

```
ol li {
    counter-increment: item;
}
```

8.3 Mistake 3: Not Using Content Property

```
/* WRONG - Counter not displayed */
li::before {
    color: blue;
}

/* RIGHT - Use content property */
li::before {
    content: counter(item);
    color: blue;
}
```

8.4 Mistake 4: Confusing Naming

```
/* WRONG - Different names */
div {
    counter-reset: chapter;
}

h1 {
    counter-increment: chapt;
}

/* RIGHT - Matching names */
div {
    counter-reset: chapter;
}

h1 {
    counter-increment: chapter;
}
```

8.5 Mistake 5: Forgetting Nested Resets

```
/* WRONG - Sections won't reset properly */
h1 {
    counter-increment: chapter;
}

h2 {
    counter-increment: section;
}
```

CSS Counter: Increment & Reset

```
/* RIGHT - Reset section counter when chapter changes */
h1 {
    counter-increment: chapter;
    counter-reset: section;
}

h2 {
    counter-increment: section;
}
```

Chapter 9

Limitations and Solutions

9.1 Limitation 1: Can't Use Counters for CSS Values

Problem: Counters work only with `content` property, not for other CSS values like width or color.

Workaround: Use CSS variables or data attributes:

```
:root {
    --main-color: blue;
}

element {
    color: var(--main-color);
}
```

9.2 Limitation 2: Counter Display Only

Problem: Counters display as text, not actual variables you can manipulate.

Workaround: Use JavaScript for more complex logic:

```
/* CSS handles simple numbering */
li::before {
    content: counter(item);
}

/* JavaScript for complex calculations */
```

9.3 Limitation 3: Scoping Issues

Problem: Counters are global within their scope.

Solution: Use descriptive names and proper reset hierarchy:

```
body {
    counter-reset: main-chapter;
}

aside {
    counter-reset: sidebar-item;
```

```
}
```

Chapter 10

Summary

[colback=lightblue, colframe=darkblue, title=What You Learned]
Key Points About CSS Counters:

1. `counter-reset` initializes counters to a value
2. `counter-increment` increases counter values
3. `counter()` displays counter values in content
4. `counters()` shows multi-level numbering
5. Support multiple counter styles (decimal, roman, alpha, etc.)
6. Perfect for automatic numbering systems
7. No HTML modification needed
8. No JavaScript required for simple numbering

Common use cases:

- Numbered lists and outlines
- Chapter and section numbering
- Figure, table, and footnote numbering
- Heading hierarchies
- Tutorial step numbering
- FAQ question numbering
- Custom list styles
- Page numbering

Master CSS counters to create sophisticated automatic numbering systems with pure CSS!

Chapter 11

Practice Exercises

11.1 Exercise 1: Simple Numbered List

Create a styled list that:

- Numbers items in circles
- Uses custom styling
- Starts from a custom number

11.2 Exercise 2: Multi-Level Outline

Build a document outline with:

- Chapter numbering (1, 2, 3)
- Section numbering (1.1, 1.2, 2.1)
- Different styles for each level

11.3 Exercise 3: Reference System

Create a document with:

- Automatic figure numbering
- Automatic table numbering
- Separate counters for each type

11.4 Exercise 4: Tutorial Steps

Build a tutorial interface with:

- Numbered steps (1, 2, 3)
- Circular badges with numbers
- Progress indication

11.5 Exercise 5: Table of Contents Style

Create heading structure that:

- Numbers chapters and sections
- Uses Roman numerals for chapters
- Uses letters for subsections

11.6 Exercise 6: Footnote References

Build a document with:

- Automatic footnote numbering
- Reference links
- Counter reset per page

Complete these exercises to master CSS counters!